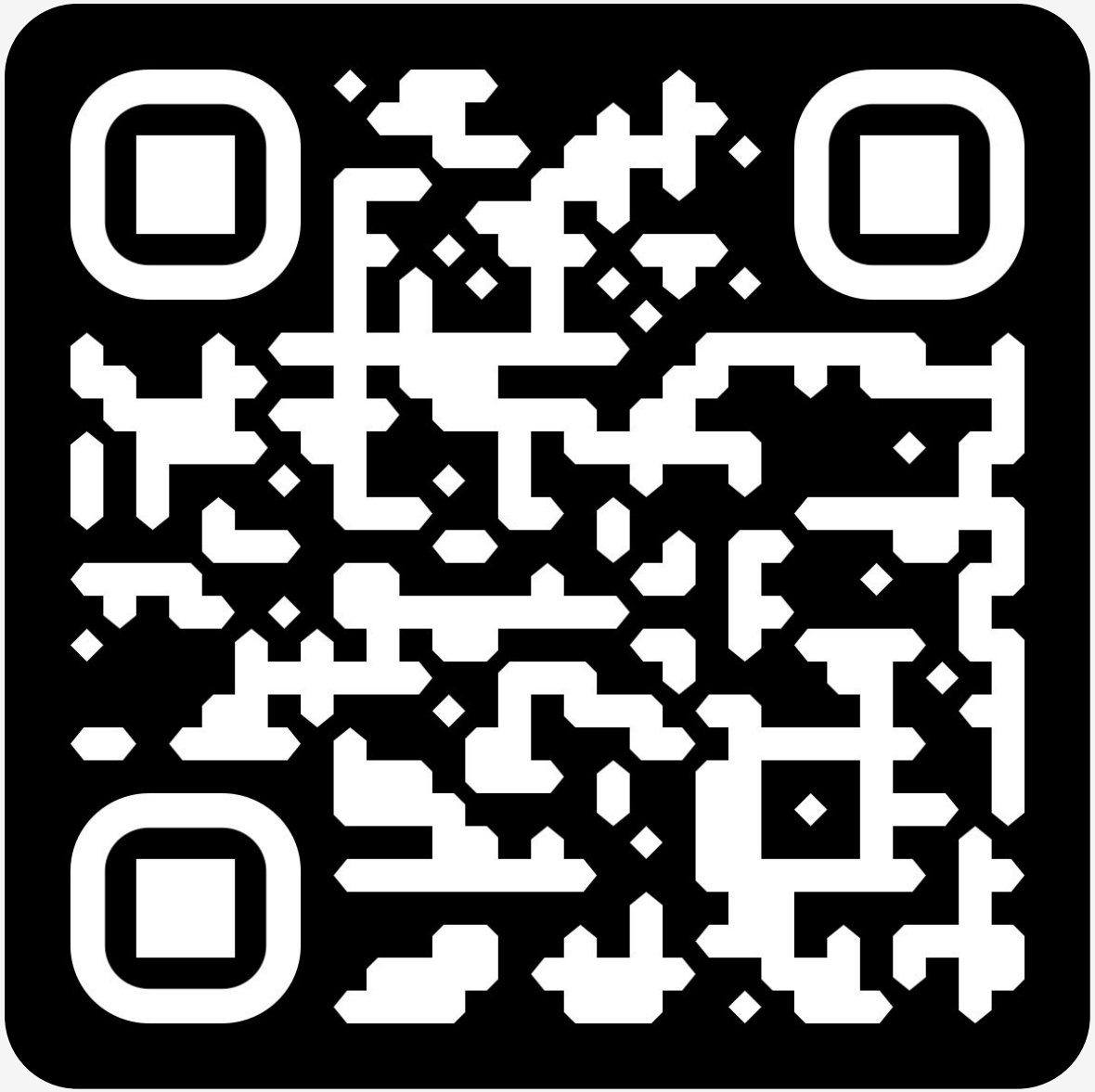
MAGNIT TECH

Лекция 1
Введение в дата-инжиниринг и архитектуры данных

Курс Data Engineering | MAGNIT TECH x НИУ ВШЭ

Чат в tg



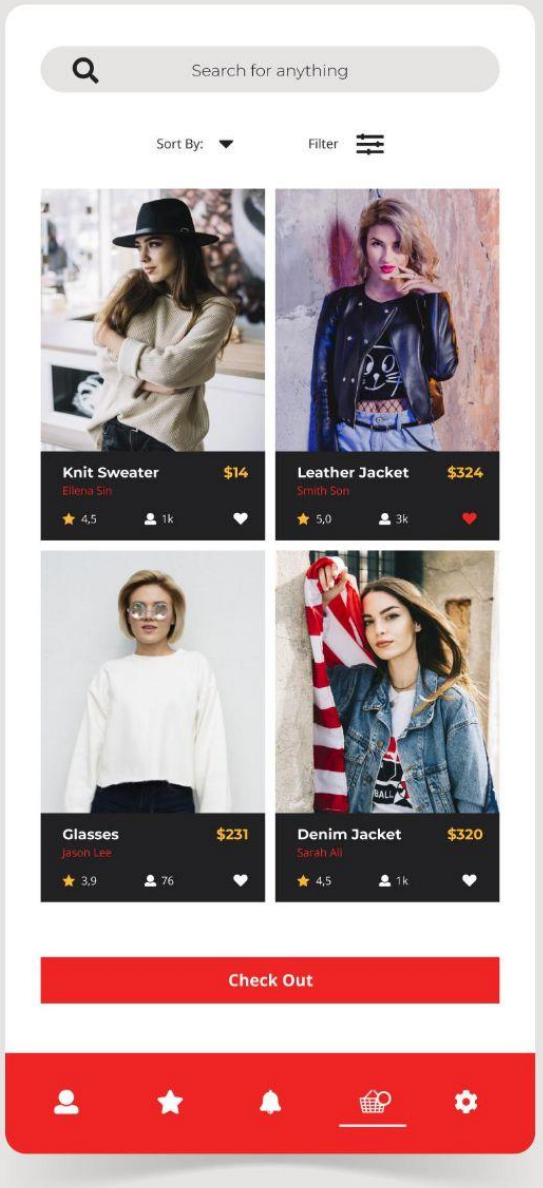
MAGNIT TECH

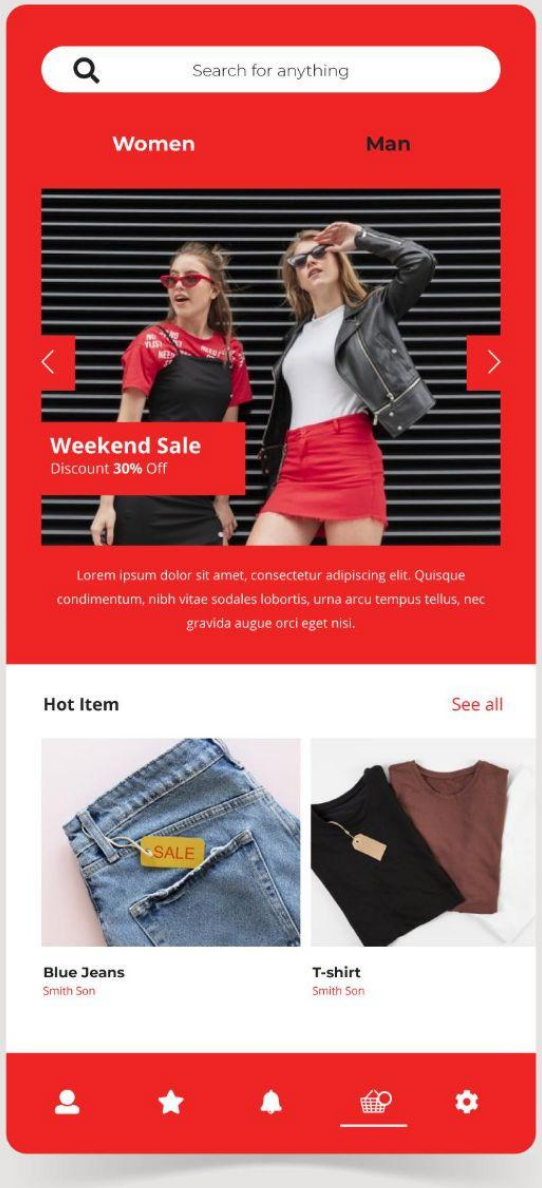
План лекции

- Роль дата-инженера и его место в команде;
- Основные этапы движения данных;
- Подходы к хранению данных: DWH, Data Lake и Lakehouse;
- Принципы ETL и ELT;
- Архитектура и инструменты курса.

MAGNIT TECH

Представим интернет-магазин





MAGNIT TECH∞

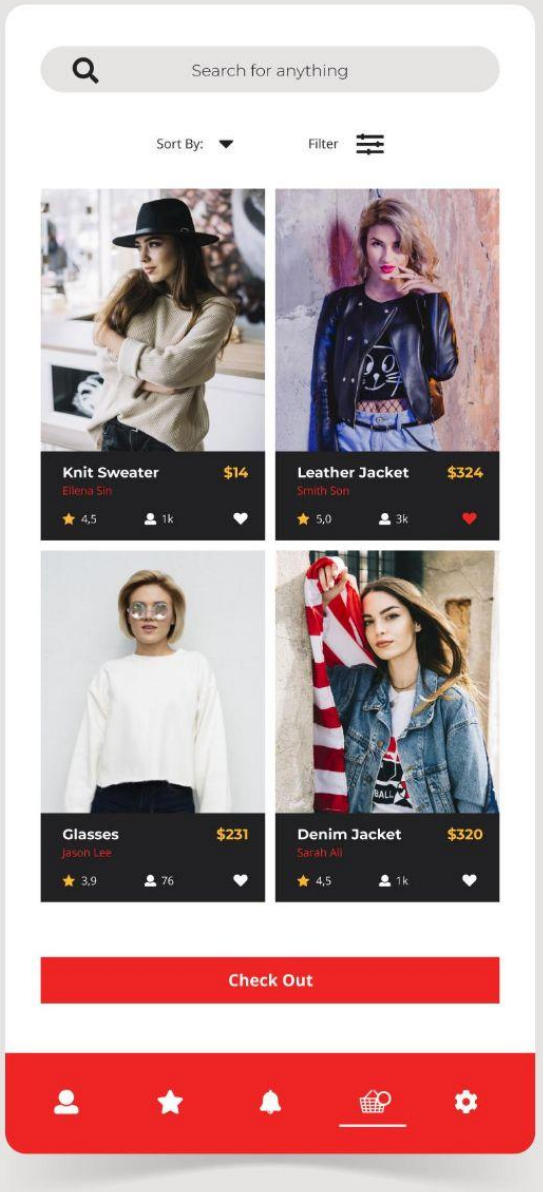
Задача инженера данных — организовать путь данных между системами так, чтобы из разрозненных технических записей получился надежный информационный продукт.

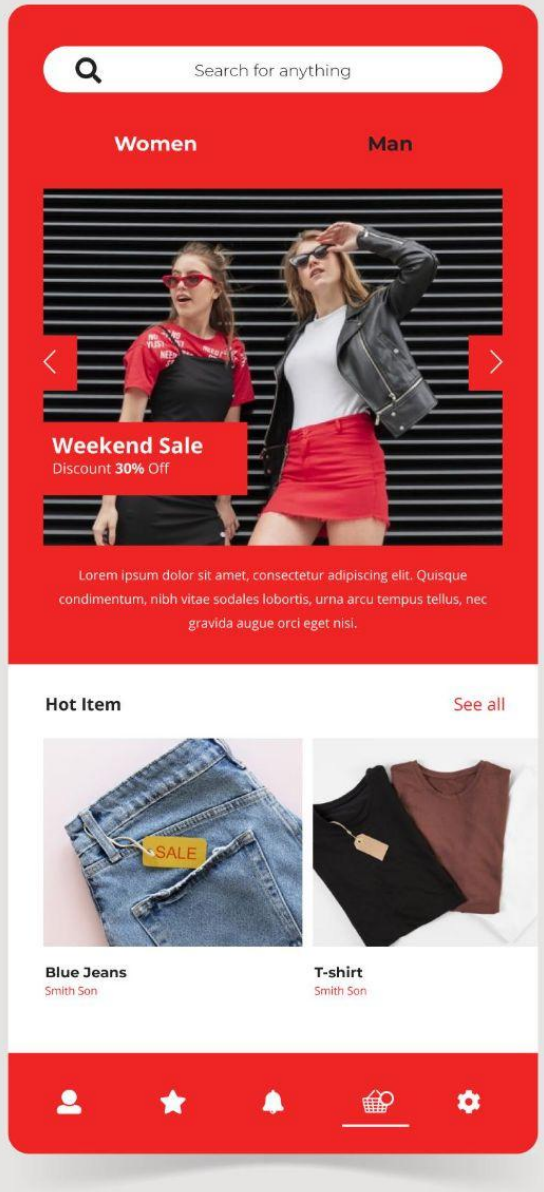
MAGNIT TECH∞

Задача инженера данных — организовать путь данных между системами так, чтобы из разрозненных технических записей получился надежный информационный продукт.

Backend отвечает за то, чтобы заказ можно было оформить. Data **Engineering** отвечает за то, чтобы спустя день, месяц и год компания могла правильно понять, что происходило с заказами.

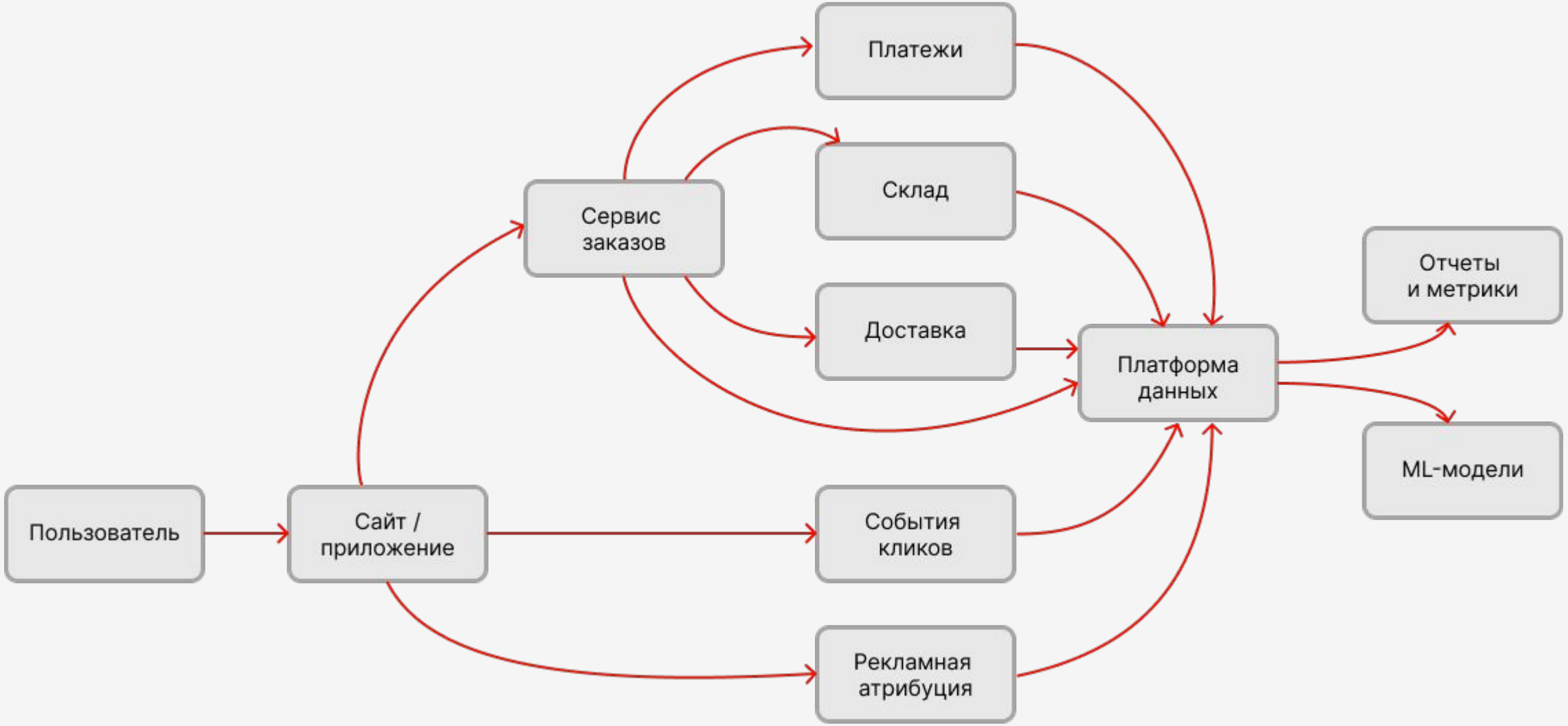
Какие **данные** понадобятся, чтобы **посчитать прибыль** интернет-магазина за вчерашний день?





MAGNIT TECH

Один заказ, много систем



Дата-инжиниринг 📌

это проектирование, разработка и ***эксплуатация*** систем, которые:

- получают данные из источников;
- надежно перемещают их;
- хранят в подходящем виде;
- очищают и преобразуют;
- предоставляют потребителям;
- контролируют качество, своевременность, безопасность и стоимость этого процесса.

Инженерная система должна переживать:

- повторный запуск;
- временное падение источника;
- опоздавшие данные;
- изменение схемы;
- дубли;
- частично выполненную загрузку;
- рост объема;
- появление новых потребителей;
- ошибку человека.

Инженерная система должна переживать:

- повторный запуск;
- временное падение источника;
- опоздавшие данные;
- изменение схемы;
- дубли;
- частично выполненную загрузку;
- рост объема;
- появление новых потребителей;
- ошибку человека.



Data pipeline — не просто последовательность преобразований.

Это последовательность преобразований **плюс правила эксплуатации.**

Какие продукты создает дата-инженер?



MAGNIT TECH

Какие продукты создает дата-инженер?

- таблица фактов продаж;
- слой сырых данных в объектном хранилище;
- ежедневная витрина для финансов;
- поток событий в Kafka;
- набор dbt-моделей с тестами и документацией;
- Spark-задача, обрабатывающая терабайты логов;
- каталог данных;
- API или SQL-интерфейс доступа к данным;
- механизм контроля качества;
- SLA по времени готовности данных.



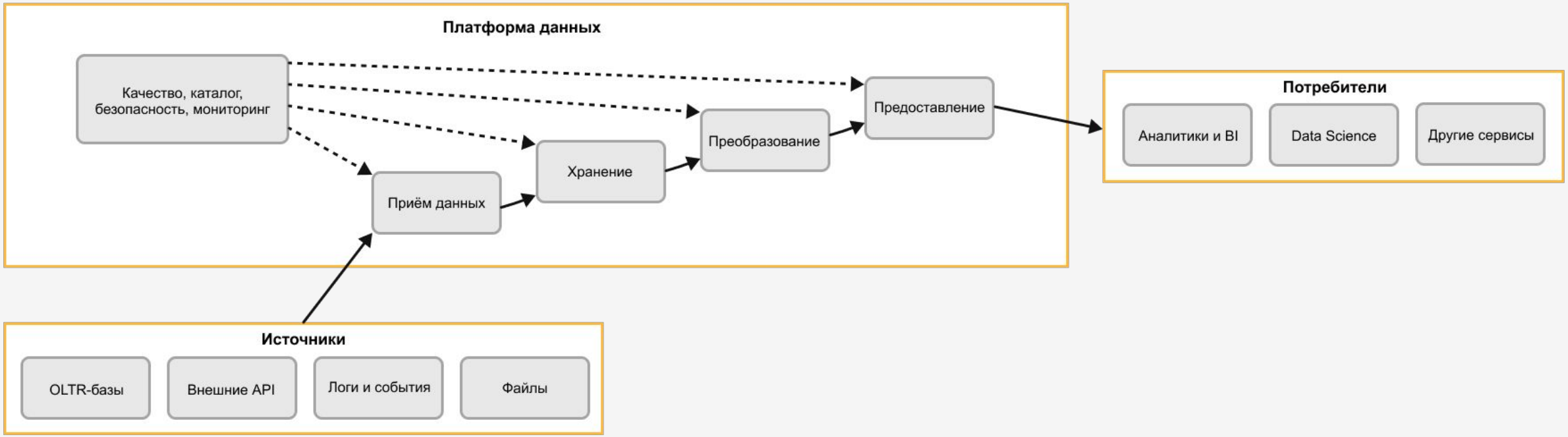
Почему нельзя «просто читать продовую базу»



Проблемы подхода

- Нагрузка
- История
- Разные источники
- Неудобная модель
- Изменчивость
- Безопасность
- Повторяемость

От источников к потребителям



Batch и streaming
как два режима движения данных

Batch

данные обрабатываются порциями. Например, раз в сутки забираем все заказы за предыдущий день.

Streaming

события обрабатываются непрерывно или почти непрерывно. Например, каждый клик пользователя сразу попадает в поток.

MAGNIT TECH∞

Необходимо классифицировать задачи:

- месячная финансовая отчетность,
- онлайн-рекомендации,
- обучение модели раз в неделю,
- обнаружение подозрительной транзакции,
- ежедневная выгрузка остатков партнеру.

MAGNIT TECH∞

Роль DE в команде



Data Engineer и Backend Engineer

Data Engineer

оптимизирует путь
и представление данных

- повторный запуск;
- забрать изменения из системы заказов;
- сохранить историю;
- объединить заказы с платежами;
- пересчитать данные после исправления ошибки;
- сделать результат доступным аналитике;
- выдержать рост объема.

Backend Engineer

оптимизирует систему под выполнение
бизнес-операций

- создать заказ;
- провести платеж;
- изменить профиль;
- вернуть пользователю ответ с небольшой задержкой;
- обеспечить корректность конкретной транзакции.

Data Engineer и Data Analyst

Data Engineer

делает так, чтобы для этих вопросов существовали надежные данные

- источники регулярно загружаются;
- поля имеют понятный смысл;
- история не теряется;
- расчет можно повторить;
- качество проверяется;
- запрос работает за приемлемое время.

Data Analyst

отвечает прежде всего на вопросы бизнеса

- почему снизилась конверсия;
- какие товары растут;
- как ведут себя разные сегменты;
- что произошло с выручкой.

Data Engineer и Data Scientist / ML Engineer

Data Scientist исследует данные и строит модели. Data Engineer обеспечивает воспроизводимое получение признаков и обучающих выборок.

ML Engineer обычно отвечает за доведение модели до production: упаковку, развертывание, инференс, мониторинг модели. Граница между DE и MLE особенно размыта вокруг feature pipelines.

Плохой сценарий:

1) Data Scientist обучил модель на локальном ноутбуке.
2) Датасет собран набором ручных действий.
3) не может точно повторить выборку через месяц.
4) В production признаки считаются иначе, чем при обучении.

DE-практики уменьшают этот разрыв: версионирование, тесты, единые преобразования, оркестрация, наблюдаемость и документированные контракты.

Data Engineer и DevOps / Platform Engineer

DevOps / Platform Engineer

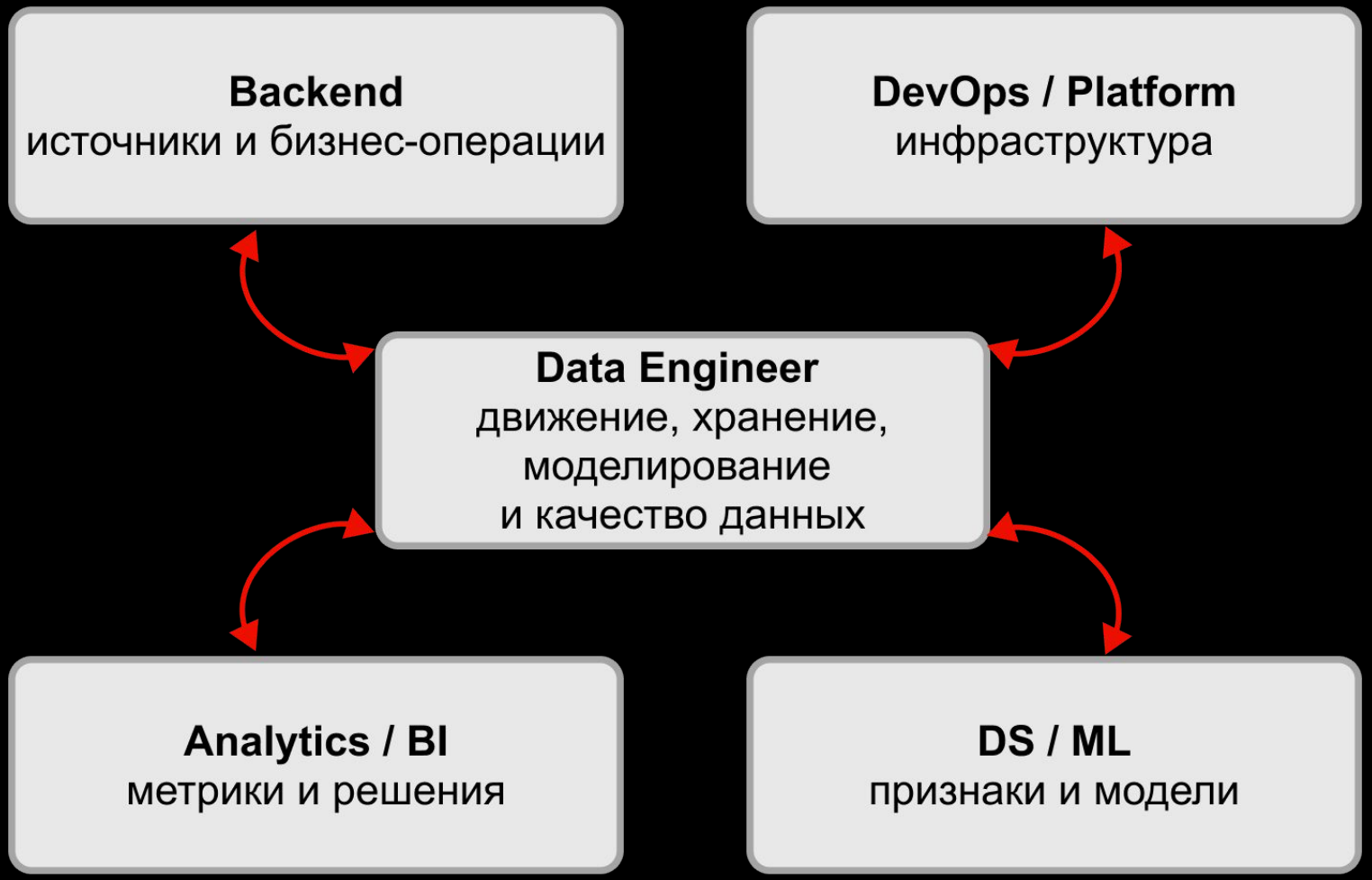
предоставляет вычислительную и инфраструктурную основу:

- Kubernetes или виртуальные машины;
- CI/CD;
- сети;
- секреты;
- мониторинг;
- Terraform;
- общие платформенные сервисы.

Data Engineer

использует эту основу для систем обработки данных и отвечает за их прикладное поведение: схемы, пайплайны, форматы, качество, партиционирование, зависимости и сроки готовности данных.

DE на стыке ролей



Как выглядит
обычная задача DE

«Каждое утро к 08:00 финансовый отчет должен содержать оплаченные и возвращенные заказы за предыдущий день»

Пример

Инженер должен выяснить:

- где находятся заказы, платежи и возвраты;
- какие поля являются ключами;
- как определить успешный платеж;
- могут ли платежи или возвраты опоздать;
- как хранить историю изменения статуса;
- что произойдет при повторном запуске;
- как обнаружить пропуск данных;
- кто получит уведомление при сбое;
- сколько времени и денег может занимать расчет;
- кто имеет право видеть персональные данные.

MAGNIT TECH

Жизненный цикл данных

Путь заказа ➡️

«Каждое утро к 08:00 финансовый отчет должен содержать оплаченные и возвращенные заказы за предыдущий день»

Пример

Инженер должен выяснить:

- где находятся заказы, платежи и возвраты;
- какие поля являются ключами;
- как определить успешный платеж;
- могут ли платежи или возвраты опоздать;
- как хранить историю изменения статуса;
- что произойдет при повторном запуске;
- как обнаружить пропуск данных;
- кто получит уведомление при сбое;
- сколько времени и денег может занимать расчет;
- кто имеет право видеть персональные данные.

Жизненный цикл данных

с контрольными точками



Свойства хорошей платформы

Надёжность

Если задача упала, мы это узнаем. После исправления ее можно безопасно перезапустить. Данные не потеряются и не продублируются. Идемпотентность

Качество

Проверять можно: уникальность ключей; отсутствие NULL в обязательных полях; допустимый диапазон значений; связи между таблицами; объем и аномалии; свежесть; специфические бизнес-инварианты.

Масштабируемость

Хорошая архитектура соответствует реальному масштабу и имеет понятный путь роста.

Наблюдаемость

Нужно отвечать на вопросы: pipeline работает или нет; когда данные обновлялись последний раз; сколько строк пришло; где возникла задержка; какие витрины затронуты; кто владелец данных.

Управляемость и безопасность

Персональные данные, платежные сведения и коммерческие показатели требуют разграничения доступа, аудита и правил хранения.

Стоимость

Любое архитектурное решение имеет цену:
хранение; вычисления;
сеть; лицензии; эксплуатация;
время инженеров.



Архитектура — это выбор компромиссов под требования, а не рейтинг технологий от старых к новым.

MAGNIT TECH

Классический Data Warehouse

Что такое DWH

Определение

Data Warehouse, или хранилище данных

централизованная система, предназначенная для аналитики и отчетности. Данные из операционных систем приводятся к согласованной структуре и загружаются в аналитическое хранилище.

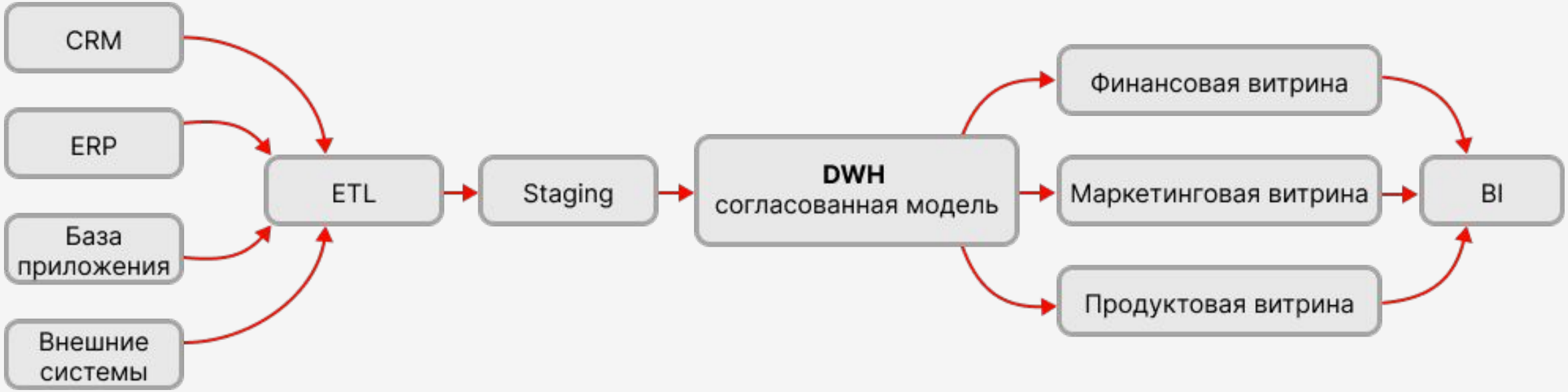
Свойства

Классическое DWH решает несколько проблем:

- объединяет данные разных источников;
- отделяет аналитику от production-систем;
- хранит историю;
- задает единые определения показателей;
- оптимизирует структуру и движок под аналитические запросы.

MAGNIT TECH

Классический Data Warehouse



Почему DWH удобен

1 Структура и контроль

Прежде чем данные станут доступны, определяется схема: типы, ключи, связи, правила. Это называют подходом schema-on-write — структура применяется при записи.Преимущество: потребитель получает уже подготовленные данные с предсказуемой семантикой.

2 Производительность SQL

Аналитические СУБД оптимизированы под сканирование и агрегацию больших наборов данных. В курсе мы отдельно сравним OLTP и OLAP, а также посмотрим на Greenplum и ClickHouse.

3 Единая версия показателей

Хранилище может зафиксировать централизованные правила:
что считать выручкой; как учитывать возвраты; к какому каналу отнести заказ; что считать активным клиентом.

❌ Ограничения классического DWH

Жесткая схема. Новый источник сначала нужно изучить и встроить в модель.

Стоимость хранения и вычислений. Особенно у традиционных проприетарных решений.

Не все данные удобны для таблиц. Логи, изображения, аудио и произвольный JSON хуже укладываются в классическую реляционную модель.

Долгий путь до доступности. При строгом ETL потребитель может ждать, пока источник полностью смоделируют.

Сложность изменений. Централизованная модель может стать узким местом большой организации.

MAGNIT TECH

Data Lake

Идея Data Lake

Определение

Data Lake

хранилище больших объемов данных в исходном или слабо обработанном виде. Современный Data Lake часто строится поверх объектного хранилища, совместимого с S3.

Свойства

В озеро можно положить:

- CSV и JSON;
- Parquet и ORC;
- логи;
- события;
- дампы баз;
- изображения;
- аудио;
- результаты ML-экспериментов.

Schema-on-read

В Data Lake часто используется schema-on-read: данные сначала сохраняются, а структура и интерпретация применяются при чтении.

Преимущества:

- быстрое подключение новых источников;
- сохранение исходных данных;
- возможность повторной обработки;
- поддержка разных форматов и сценариев;
- независимое масштабирование хранения и вычислений.

Слои данных

В Data Lake часто используется schema-on-read: данные сначала сохраняются, а структура и интерпретация применяются при чтении.

Одна из возможных моделей

- Raw / Bronze: данные максимально близки к источнику;
- Clean / Silver: очищенные, типизированные и дедуплицированные данные;
- Curated / Gold: бизнес-сущности и готовые наборы для потребления.

Названия слоев могут отличаться. Важен не цвет, а контракт:

- что разрешено в слое;
- какие гарантии он дает;
- кто им пользуется;
- можно ли его пересобрать.

Слои Data Lake



Почему Data Lake может превратиться в Data Swamp

Если просто складывать файлы без правил, возникает «болото данных»:

- непонятно, какие данные актуальны;
- неизвестно, кто владелец;
- схемы не документированы;
- есть дубли;
- права доступа настроены хаотично;
- один набор нельзя связать с другим;
- тысячи мелких файлов замедляют обработку;
- никто не уверен, можно ли удалять старые данные.

Чтобы озеро оставалось платформой, нужны:

- каталог и метаданные;
- стандарты именования;
- управление схемами;
- форматы и партиционирование;
- контроль качества;
- разграничение доступа;
- политики жизненного цикла;
- lineage;
- владельцы наборов данных.

MAGNIT TECH

Lakehouse

Идея Lakehouse

DWH дает удобные таблицы, транзакции, SQL и управляемость, но может быть дорогим и менее гибким для сырых данных.

Data Lake дает дешевое масштабируемое хранение и свободу форматов, но без дополнительного слоя плохо обеспечивает надежные изменения таблиц и удобство работы потребителей.

Lakehouse пытается совместить:

- хранение данных в объектном хранилище;
- открытые колоночные форматы;
- табличные метаданные;
- транзакционные гарантии;
- эволюцию схемы;
- SQL-доступ;
- работу нескольких вычислительных движков.

Почему папки с Parquet еще не Lakehouse

Если мы просто положили orders.parquet в S3, появляются вопросы:

- какие файлы сейчас входят в таблицу;
- как атомарно заменить часть данных;
- что увидит читатель во время перезаписи;
- как удалить или обновить строки;
- как вернуться к предыдущей версии;
- как безопасно менять схему;
- как несколько процессов будут писать одновременно.

Табличные форматы, например Apache Iceberg, Delta Lake или Apache Hudi, добавляют слой метаданных и протокол управления таблицей поверх файлов.

Файлы содержат данные, а табличный формат содержит правила и метаданные, которые превращают набор файлов в управляемую таблицу.

Lakehouse как слои



DWH, Lake и Lakehouse — не линейная эволюция

Свойство	DWH	Data Lake	Lakehouse
Основная цель	Управляемая аналитика	Масштабное гибкое хранение	Аналитические таблицы поверх lake
Данные	В основном структурированные	Любые	В основном структурированные и полуструктурированные
Схема	Чаще schema-on-write	Чаще schema-on-read	Оба подхода
Хранение	Внутри аналитической системы	Объектное хранилище	Объектное хранилище
SQL	Нативный и удобный	Зависит от движка	Через совместимые движки
Обновления таблиц	Обычно нативные	Сложны на уровне файлов	Через табличный формат
Типичный риск	Жесткость и стоимость	Data swamp	Сложность экосистемы

MAGNIT TECH

Рассмотрим три задачи

1) Регламентированная финансовая отчетность с жесткими правилами.
2) Хранение всех сырых логов за несколько лет.
3) Большие аналитические таблицы в S3, которые читают Spark и Trino и которые нужно версионировать.

MAGNIT TECH

ETL и ELT

Разница в порядке операций

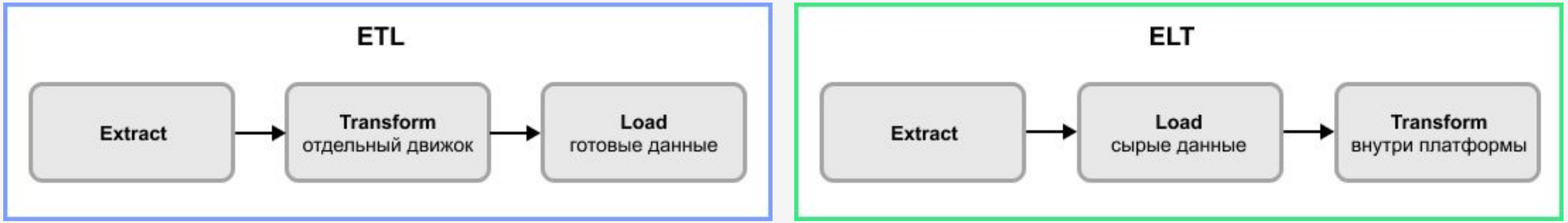
ETL: Extract → Transform → Load

- Извлекаем данные.
- Преобразуем во внешнем вычислительном слое.
- Загружаем готовый результат в целевую систему.

ELT: Extract → Load → Transform

- Извлекаем данные.
- Загружаем их в целевую платформу.
- Преобразуем уже средствами этой платформы.

ETL и ELT


ETL
Extract
Transform
отдельный движок
Load
готовые данные
ELT
Extract
Load
сырые данные
Transform
внутри платформы

Почему исторически был распространен ETL и почему стал популярен ELT



Традиционные DWH были дорогими и ограниченными по ресурсам. Загружать в них все сырые данные было невыгодно. Данные очищали и агрегировали до загрузки.

ETL естественен, когда:

- до целевой системы нельзя допускать чувствительные поля;
- формат источника требует сложной предварительной обработки;
- целевая система принимает только подготовленную структуру;
- вычисление удобнее выполнить специализированным движком;
- объем нужно сильно сократить до передачи.

Почему исторически был распространен ETL и почему стал популярен ELT

Современные аналитические платформы получили мощные и масштабируемые вычисления. Объектное хранение стало относительно дешевым. Стало возможно быстро загрузить сырой слой, сохранить историю, а преобразования описывать SQL-кодом внутри платформы.

Преимущества ELT

- исходные данные доступны для повторной обработки;
- быстрее подключается новый источник;
- SQL-трансформации ближе к аналитикам;
- проще разделять слои staging, intermediate и marts;
- вычисления выполняются там, где лежат данные;
- dbt позволяет оформлять преобразования как код с тестами и документацией.

ETL и ELT

Реальный pipeline может сочетать оба подхода.

Пример

1) События извлекаются из источника.
2) Персональные данные маскируются до передачи — элемент ETL.
3) Очищенные сырые события загружаются в S3.
4) Spark преобразует их в Parquet.
5) dbt строит витрины внутри аналитической СУБД — ELT.

Вместо вопроса «Что лучше?» задавать:

- где дешевле и удобнее выполнять вычисление;
- нужно ли сохранять сырой слой;
- можно ли загружать исходные данные с точки зрения безопасности;
- какие объемы передаются;
- какие возможности есть у целевой платформы;
- кто будет поддерживать преобразования;
- как переиграть историю после изменения логики.

MAGNIT TECH

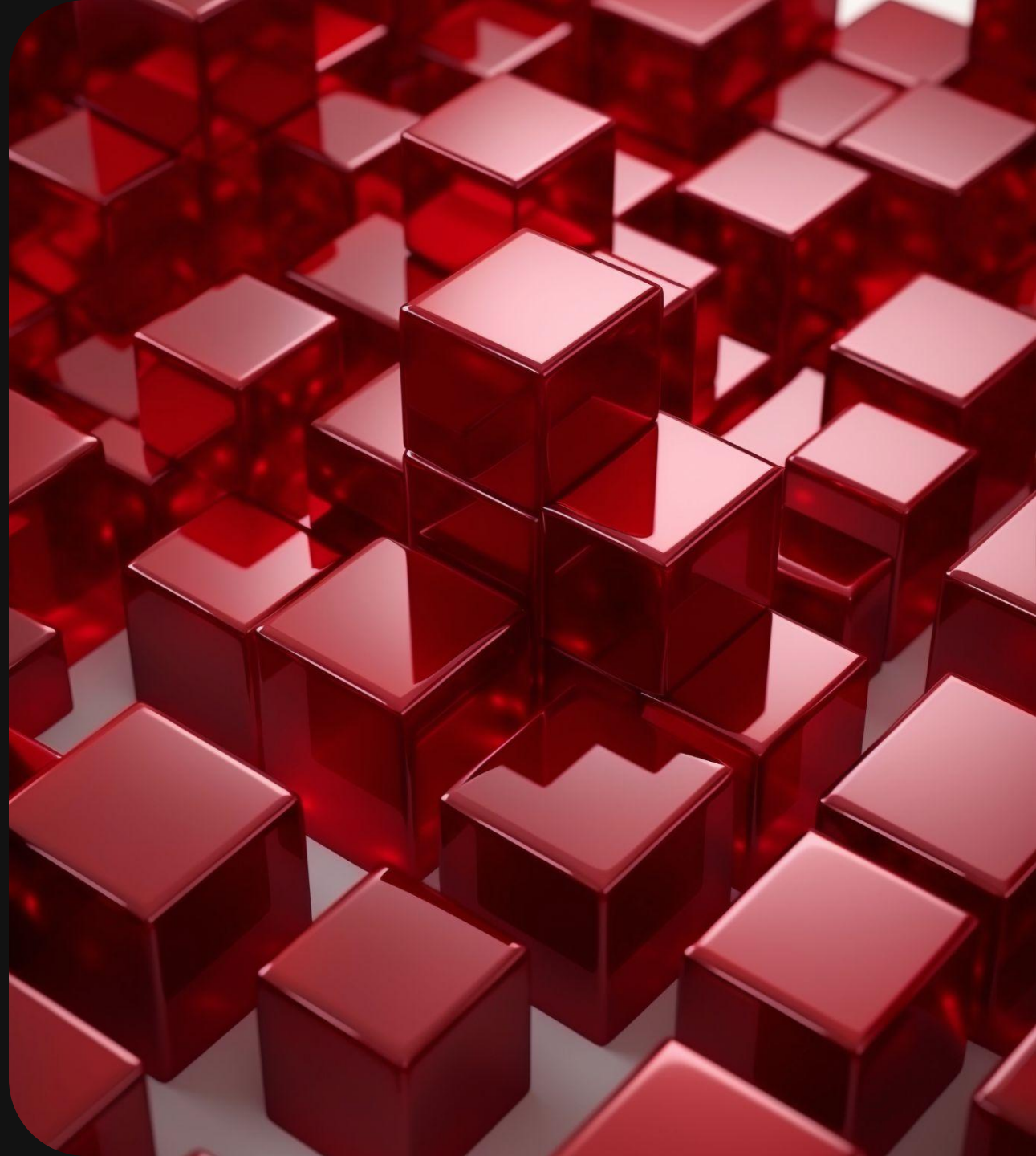
Рассмотрим кейс

Компания получает небольшой CSV со справочником валют один раз в день. Его удобно очистить до загрузки — ETL может быть достаточен.

Компания получает миллиарды событий кликов, хочет хранить историю и строить новые модели поведения. Логично сначала надежно принять сырой поток, а затем строить разные преобразования — ближе к ELT/lake-подходу.

MAGNIT TECH∞

Карта курса и сквозной проект



Что будем строить

*На протяжении курса каждый студент собирает **дата-платформу интернет-магазина**. Не набор независимых лабораторных, а одну систему, которая постепенно обрастает слоями.*

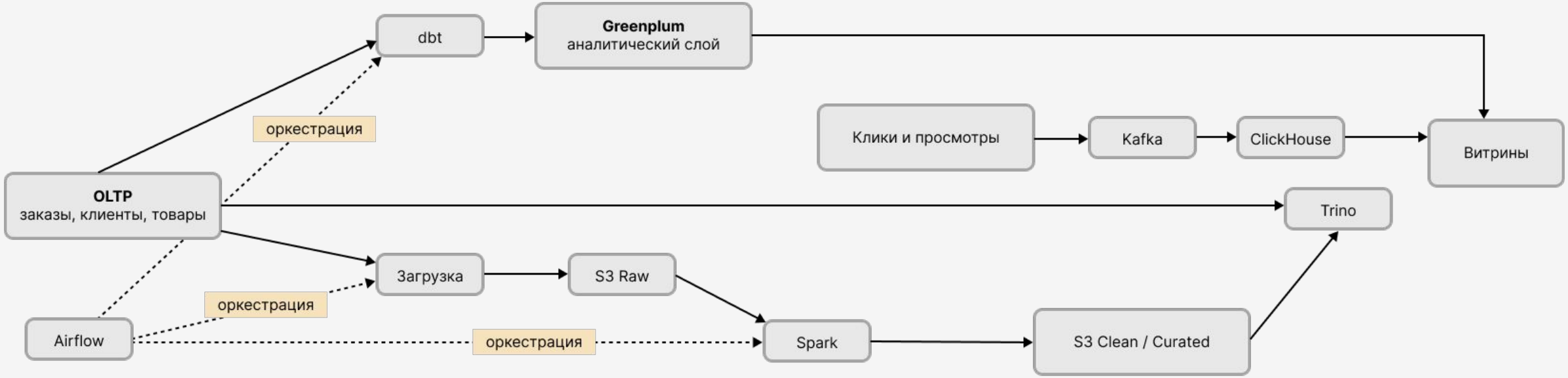
Какой путь?

1) Получаем данные операционной базы интернет-магазина.
2) Учимся уверенно преобразовывать их SQL.
3) Разбираемся, чем OLTP отличается от OLAP и где нужны Greenplum и ClickHouse.
4) Проектируем факты, измерения, звезду, снежинку и историчность.
5) Оформляем SQL-трансформации в dbt: модели, тесты, документация, макросы.
6) Храним сырой слой в S3, выбираем форматы и партиционирование.
7) Обрабатываем большие объемы в Spark.
8) Читаем разные источники через Trino.
9) Добавляем поток событий через Kafka.
10) Оркестрируем зависимости и расписания в Airflow.

Роль каждого инструмента

Инструмент	Вопрос, на который он отвечает
SQL	Как выразить преобразование и получить нужный набор данных?
Greenplum	Как выполнять аналитический SQL в MPP-реляционной системе?
ClickHouse	Как быстро агрегировать большие объемы колоночных данных?
dbt	Как превратить SQL-трансформации в тестируемый и документированный проект?
S3	Где дешево и масштабируемо хранить файлы и сырой слой?
Parquet / ORC / Avro	В каком физическом формате хранить данные?
Spark	Как распределенно обрабатывать большие объемы?
Trino	Как выполнять SQL-запросы к разным источникам и файловому lake?
Kafka	Как надежно передавать поток событий?
Airflow	Как запускать задачи по расписанию и управлять их зависимостями?

Итоговая архитектура проекта курса



Несколько тезисов в завершение

- Данные компании распределены по множеству систем, а значимые ответы требуют их объединения.
- Data Engineer отвечает за надежный путь данных от источника до потребителя.
- DWH, Data Lake и Lakehouse дают разные компромиссы хранения, структуры и обработки.
- ETL и ELT — варианты организации преобразований, выбор зависит от контекста.
- Инструмент нужно выбирать после требования, а не наоборот.

MAGNIT TECH

Спасибо за занятие!

